```
  def received(order)
    @order = order

    mail to: order.email, subject: "Pragmatic Store Order Confirmation"
  end
```

What we did here is add `order` as an argument to the method-received call, add code to copy the parameter passed into an instance variable, and update the call to `mail` specifying where to send the email and what subject line to use.

Generating Emails

Now that we have our template set up and our mailer method defined, we can use them in our regular controllers to create and/or send emails. Note that just calling the method we defined isn't enough; we also need to tell Rails to actually send the email. The reason this doesn't happen automatically is that Rails can't be 100 percent sure if you want to deliver the email right this moment, while the user waits, or later, in a background job.

Generally, you don't want the user to have to wait for emails to get sent, because this can take a while. Instead, we'll send it in a background job (which we'll learn more about later in the chapter) by calling `deliver_later` (to send the email right now, you'd use `deliver_now`).[73]

rails80/depot_qa/app/controllers/orders_controller.rb

```
    def create
      @order = Order.new(order_params)
      @order.add_line_items_from_cart(@cart)

      respond_to do |format|
        if @order.save
          Cart.destroy(session[:cart_id])
          session[:cart_id] = nil
»         OrderMailer.received(@order).deliver_later
          format.html { redirect_to store_index_url, notice:
            "Thank you for your order." }
```